

Confirmar compras, costo congelado y cajas numeradas

Que la mercancia entre al almacen solo cuando alguien lo confirma, que la utilidad use el costo real de lo vendido y que cada caja se pueda identificar por su numero.

El problema que resolvio. Registrar una compra suma stock al instante. Eso no es cierto: la mercancia todavia esta en camino. Si el proveedor no la entrega, el inventario quedo inflado y nadie lo sabe.

Lo que cambio. El efecto en el inventario se mudo de *crear* a *confirmar*. Comprar sin confirmar no toca el stock y se puede borrar; confirmar lo mete al almacen y lo congela.

Utilidad real. Nuevo reporte: ingresos menos el costo de lo vendido, con el costo congelado en cada renglón de venta. No es menos compras del mes.

Cajas numeradas. El vendedor escribe el numero de la caja y despues se puede filtrar el reporte por ella, con sus ventas y su utilidad.

Un bug de paso. Las ventas anuladas se estaban contando como ingresos desde V2. Tambien se corrigio.

Verificacion. 173 tests en verde, frontend compilado, 5 endpoints y 1 permiso nuevos.

Lo que pidio el dono fueron tres pedidos que parece que no tienen relacion: un boton de confirmar en Compras, reportes de cuanto se gano, y cajas con numero. Pero los tres son la **misma regla: nada se contabiliza hasta que pasa por un acto explicito de una persona.**

Comprar no suma stock hasta que alguien confirma que la caja llevo. La utilidad no se calcula con el costo de hoy sino con el costo que tenia el producto *cuando se vendio*. Y una caja no se identifica por un id que genera la base de datos sino por el numero que escribe el vendedor. En los tres casos, el datoAutomatico cede el paso al dato que el humano declaro.

Las 3 decisiones que tomo el dono

Decision	Elegido	La alternativa que se descarto
Cajas	Una sola caja activa, con numero	Varias cajas simultaneas, una por vendedor (hubo que agregar user_id)
Utilidad	Costo real de lo vendido	Ventas menos las compras del periodo
Cancelar compra pendiente	Se elimina (no hay nada que revertir)	Se conservaba con estado cancelada, como la venta
Quien confirma	Permiso nuevo CONFIRMAR_COMPRAS (ADMIN y ALMACENISTA)	Reusar CREAR_COMPRAS o CANCELAR_COMPRAS

La unica que cambio el modelo de datos de verdad fue la primera: sin user_id la caja se identifica por numero, no por quien la abrio.

Antes, create() armaba la compra y de paso sumaba el stock y guardaba el costo. Ahora create() solo deja constancia del documento, y hay un metodo nuevo, confirm(), que es el que hace que la mercancia entre al almacen.

Estado	Que significa	Toca el inventario	Se puede cancelar
PENDIENTE	Registrada, en camino	No	Si, se borra
CONFIRMADA	La mercancia llego	Si, de una vez	No, 409

Registrar y confirmar no es lo mismo por una razon logistica, no por formalismo: una compra puede cancelarse porque el proveedor no la entrego, y en ese caso el sistema no debe tener mercancia que nunca existio.

create() ya no toca el inventario

service/impl/PurchaseImpl.java - create()

```
@Override
@Transactional(readOnly = true)
public List<PurchaseDTO> getAll(){
    return purchaseRepository.findAllByOrderByPurchaseDateDesc().stream()
        .map(PurchaseMapper::toDto)
        .toList();
}

//Compras de un proveedor
@Override
@Transactional(readOnly = true)
public List<PurchaseDTO> getByProvider(Long providerId){
    if(providerId != null){
        providerRepository.findById(providerId)
            .orElseThrow(() -> new PurchaseException("El proveedor indicado no existe", HttpStatus.NOT_FOUND));
    }
}
```

Lo que quito no fue una validacion, fue un **efecto secundario**. Ese es el punto: mientras el inventario se escribiera al registrar, *cancelar* tenia que deshacerlo, y deshacer una suma de stock es donde nacio el bug del Math.max.

confirm(): el metodo que si suma el stock

service/impl/PurchaseImpl.java - confirm()

```
}
if(item.getQuantity() == null || item.getQuantity() <= 0){
    throw new IllegalArgumentException("La cantidad debe ser mayor a 0");
}
if(item.getUnitCost() != null && item.getUnitCost().signum() < 0){
    throw new IllegalArgumentException("El costo unitario no puede ser negativo");
}
}
}
```

- **Es idempotente.** Si ya estaba confirmada devuelve 200 y no vuelve a sumar. El doble clic y el reintento tras un corte de red son el caso normal en un POS, y un 409 ahi seria confuso: el usuario pensaria 'acabo de confirmar, que le pasa?'. Lo que no puede pasar es sumar dos veces, y por eso el if(!purchase.isConfirmed()) existe.
- **Escribe cost, nunca price.** El costo es lo que se le pago al proveedor y solo el modulo de compras lo conoce. El precio de venta es una decision comercial: si se actualizara con el costo, cada compra borrarla el margen que el dono eligio y el producto se venderia a precio de costo.
- **El costo no se revierte nunca.** No hay ruta que lo revierta, igual que en las ventas (V2): es el ultimo costo de compra vigente.

cancel() se simplifico hasta casi desaparecer

```

service/impl/PurchaseImpl.java - cancel()
    if(purchase.getDetails() != null){
        for(PurchaseDetailEntity detail : purchase.getDetails()){
            ProductEntity product = productRepository.findById(detail.getProduct().getId())
                .orElseThrow(() -> new PurchaseException(
                    "El producto asociado a la compra ya no existe; no se puede confirmar",
                    HttpStatus.CONFLICT));

            //1) Entra la mercancía al almacén.
            product.setStock(product.getStock() + detail.getQuantity());

            // !? AQUÍ ESTÁ LA DUDA FRECUENTE: se actualiza `cost` y NO `price`.
            // Son dos columnas distintas, y a propósito:
            //
            // cost = lo que le PAGAMOS al proveedor. Solo el módulo de
            //        compras lo escribe, porque solo él sabe lo que se
            //        pagó. Alimenta `GET /api/local/reports/margins`
            //        (margen = price - cost) y, desde V3, la utilidad.
            // price = lo que le COBRAMOS al cliente. Es una decisión
            //        comercial (margen, competencia, promociones) y NO
            //        debe cambiar solo porque el proveedor cobró más caro.
            //
            // Si se actualizara `price` con el costo, cada compra borraría el
            // margen que el dueño eligió y el producto se vendería a precio
            // de costo. Recalcular el precio es una REGLA DE NEGOCIO
            // (p.ej. "nuevo costo + X% de margen") y va en el CRUD de
            // productos, no de paso aquí.
            product.setCost(detail.getUnitCost());
            productRepository.save(product);
        }
    }

    purchase.setConfirmed(true);
    purchase.setConfirmedAt(java.time.LocalDateTime.now());

    return PurchaseMapper.toDto(purchaseRepository.save(purchase));

```

El metodo entero son 20 lineas porque ya no hay nada que hacer. Antes tenia dos fases, acumulaba conflictos en una lista y abortaba con 409. Todo eso existia para **no escribir un numero falso** en el stock. Con el efecto mudado a confirmar, una compra pendiente nunca toco el inventario, asi que borrarla es suficiente.

El endpoint y su permiso

```

controller/PurchaseController.java - PATCH /{id}/confirm
    * sumar el stock (doble clic o reintento tras corte de red).
    */
    @PreAuthorize("hasAuthority('CONFIRMAR_COMPRAS')")
    @PatchMapping("/{id}/confirm")
    public ResponseEntity<PurchaseDTO> confirm(@PathVariable Long id){
        return ResponseEntity.ok(purchaseService.confirm(id));
    }

    //Cancelar una compra PENDIENTE (la borra; 409 si ya está confirmada)
    @PreAuthorize("hasAuthority('CANCELAR_COMPRAS')")
    @DeleteMapping("/{id}")
    public ResponseEntity<Void> cancel(@PathVariable Long id){
        purchaseService.cancel(id);
        return ResponseEntity.noContent().build();
    }

```

Es **PATCH** y no PUT porque es un cambio parcial de estado. Un PUT reenviaría la compra entera y podría pisar el total o los renglones. El permiso CONFIRMAR_COMPRAS es propio: quien registra una compra no debería poder, solo por eso, dar entrada a la mercancía. Se asigna a **ADMIN y ALMACENISTA** (no a CAJERO) porque confirmar

es una operacion de almacen: es quien recibe y cuenta la caja.

Tercera migracion del proyecto, 242 lineas. Hace cinco cosas, en este orden porque el orden importa: primero agrega columnas, despues las rellena, y solo al final las restringe.

3.1 Las compras: el backfill que hace el trabajo pesado

```
V3_compras_confirmadas_costo_y_cajas.sql - purchases
-- Backfill: TODAS las compras que ya existian se marcan CONFIRMADAS.
--
-- Antes de V3 crear la compra ya sumaba el stock, asi que su mercaderia esta
-- (o estuvo) en el almacen y su costo ya es el vigente. Dejarlas como
-- pendientes seria un error de DATOS: permitiria borrar una compra que ya sumo
-- stock, y el inventario quedaria con existencias de fantasma. Es el mismo
-- criterio que en V2 con las ventas: lo que ya ocurrio no se "desconfirma".
--
-- confirmed_at = purchase_date (no NOW()) para que la fecha de confirmacion sea
-- la REAL y no la de la migracion;asi el historico de caja sigue siendo creible.
-- COALESCE protege el caso purchase_date IS NULL: en vez de dejar esa compra
-- colgada en PENDIENTE (que permitiria borrar un stock ya aplicado) se le pone
-- la fecha de la migracion.
UPDATE public.purchases
SET confirmed = true,
    confirmed_at = COALESCE(purchase_date, CURRENT_TIMESTAMP)
WHERE confirmed = false;

COMMENT ON COLUMN public.purchases.confirmed IS
    'true = compra confirmada: el stock ya sumo y el costo ya se guardo. NO se puede cancelar';
COMMENT ON COLUMN public.purchases.confirmed_at IS
    'Momento en que se confirmo la mercaderia y se aplico el stock. NULL mientras siga pendiente';
```

Todas las compras que ya existian se marcan confirmadas. No es una conveniencia: es la unica opcion correcta. Antes de V3, crear una compra sumaba el stock, así que la mercancía *esta o estuvo* en el almacen. Dejarlas como pendientes permitiría borrar una compra que ya había sumado stock, y el inventario quedaría con existencias de fantasma: el mismo bug del 2026-09-30, pero dejando que ocurra por otra puerta.

confirmed_at usa COALESCE(purchase_date, CURRENT_TIMESTAMP) en vez de NOW() a secas: así la fecha de confirmacion es la REAL y no la de la migracion, y el historico sigue siendo creible. El COALESCE cubre el caso de una compra sin fecha en lugar de dejarla colgada en pendiente.

3.2 El costo congelado de la venta

```
V3_compras_confirmadas_costo_y_cajas.sql - sale_details.unit_cost
ALTER TABLE public.sale_details
    ADD COLUMN IF NOT EXISTS unit_cost numeric(38,2);

UPDATE public.sale_details d
SET unit_cost = p.cost
FROM public.products p
WHERE d.product_id = p.id
    AND d.unit_cost IS NULL;

COMMENT ON COLUMN public.sale_details.unit_cost IS
    'Costo del producto congelado al momento de la venta. Base del reporte de utilidad';
```

Se agrega unit_cost al renglón de venta y el backfill lo rellena con el **costo actual** del producto. Es la mejor aproximacion disponible, no el dato historico exacto: nadie guardaba el costo al vender. Por eso la columna es nullable, y por eso el reporte cuenta cuantos renglones quedaron sin costo.

3.3 Cajas numeradas, y por que UNIQUE

V3_compras_confirmadas_costo_y_cajas.sql - cash_registers.number

```
ALTER TABLE public.cash_registers
  ADD COLUMN IF NOT EXISTS number varchar(50);

UPDATE public.cash_registers
SET number = 'CAJA-' || id
WHERE number IS NULL
  OR btrim(number) = '';

ALTER TABLE public.cash_registers
  ALTER COLUMN number SET NOT NULL;
```

- **NULL, relleno, y despues NOT NULL.** En PostgreSQL el default de una columna nueva *no* aplica a las filas que ya existian, asi que no se puede agregar la columna NOT NULL de una vez en una tabla con datos. Se agrega nullable, se rellena, y solo entomo se restringe.
- **Backfill 'CAJA-' || id.** Es unico por construccion (el id lo es) y ademas deja claro que esas cajas NO fueron numeradas por un humano.
- **UNIQUE, no por capricho.** El numero es lo que permite filtrar reportes. Si dos cajas pudieran compartirlo, el filtro juntaria dos cortes distintos en una misma fila, y el reporte diria algo que no es cierto.

3.4 El permiso: por que en la migracion y no en el bootstrap

V3_compras_confirmadas_costo_y_cajas.sql - seed idempotente

```
INSERT INTO public.permissions (name)
SELECT 'CONFIRMAR_COMPRAS'
WHERE NOT EXISTS (SELECT 1 FROM public.permissions WHERE name = 'CONFIRMAR_COMPRAS');

INSERT INTO public.role_permissions (role_id, permission_id)
SELECT r.id, p.id
FROM public.roles r
CROSS JOIN public.permissions p
WHERE r.name IN ('ADMIN', 'ALMACENISTA')
  AND p.name = 'CONFIRMAR_COMPRAS'
  AND NOT EXISTS (
    SELECT 1
    FROM public.role_permissions rp
    WHERE rp.role_id = r.id
      AND rp.permission_id = p.id
  );
```

El seed va en la migracion y no en RolePermissionBootstrap por una decision que ya se habia tomado el 17 de septiembre: ese bootstrap siembra la primera vez y despues no vuelve a tocar nada (no es self-healing). Si el permiso se asignara ahi, en una base ya sembrada **nunca llegaria** a los roles. En la migracion es explicito, versionado y corre una vez. Es idempotente (WHERE NOT EXISTS), asi que no falla si el bootstrap ya habia insertado la fila.

El CHECK permissions_name_check se reescribe con los 37 nombres. En una base nueva Flyway aplica V1, V2 y V3 en orden y el resultado es correcto; en una existente hay que aplicar V2 y V3, que es lo que hacen solas. El enum de Java y las tres migraciones tienen que seguir sincronizados.

La pregunta del dono era 'cuanto gana el negocio'. La respuesta ingenua es restar las compras del mes a las ventas del mes. **Esa formula esta mal**, y el error vale dinero real.

La formula correcta es **ingresos menos costo de lo vendido**: lo que costo exactamente lo que los clientes se llevaron. Y para eso el costo tiene que ser el de aquel momento, por lo que se congela en el renglón.

```
repository/SaleRepository.java - sumCostOfGoodsSold()
    SELECT COALESCE(SUM(d.quantity * COALESCE(d.unitCost, 0)), 0),
           COUNT(CASE WHEN d.unitCost IS NULL THEN 1 END),
           COALESCE(SUM(d.quantity), 0)
    FROM SaleDetailEntity d JOIN d.sale s
    WHERE s.paymentStatus = :status AND s.cancelled = false
           AND s.saleDate >= :from AND s.saleDate < :to
    """
    List<Object[]> sumCostOfGoodsSold(
        @Param("from") LocalDateTime from,
        @Param("to") LocalDateTime to,
        @Param("status") PaymentStatus status);

    /**
     * Ventas de una caja concreta (V3), para el detalle "que se vendio en esta
     * caja". Filtra las anuladas igual que el resto de reportes.
     */
    @Query("""
        SELECT s
        FROM SaleEntity s
        WHERE s.cashRegister.id = :cashId AND s.cancelled = false
        ORDER BY s.saleDate DESC
    """)
```

Campo	Que es	De donde sale
revenue	Ingresos del periodo	Suma de los totales de las ventas validas
costOfGoodsSold	Lo que costo lo vendido	quantity x unitCost de los renglones
grossProfit	Utilidad bruta	revenue - costOfGoodsSold
marginPercent	Margen sobre venta	grossProfit / revenue x 100
itemsWithoutCost	Renglones sin costo	Renglones con unitCost NULL (para avisar)

El margen se calcula sobre **venta**, que es como se mide el markup de un comercio. Cuando no hubo ingresos en el periodo, el margen es 0 en vez de una division entre cero.

El calculo vive en ReportsImpl.getProfit() y hace dos consultas separadas: una por los totales de venta y otra por el costo. Se podrian haber hecho en una sola con un JOIN, pero separarlas mantiene el patron del resto del modulo (agregar en SQL, transformar en Java) y hace que cada consulta se pueda leer y verificar por separado.

Una caja inexistente devuelve 404, no 400: el recurso no existe. Para eso se creo ReportException, que transporta su propio HttpStatus como ya hacian SaleException y PurchaseException. El servicio declara el codigo; el handler solo lo traduce.

El bug que aparecio por el camino

Al agregar el reporte de utilidad se hizo obvio algo que V2 habia destapado sin que nadie lo notara. En V2, anular una venta dejo de **borrar** la fila (es evidencia conptable: un contador que baja solo es un agujero de fraude) pero **no cambio su paymentStatus**, que sigue en APPROVED. Los agregados filtraban solo por APPROVED, así que **una venta anulada seguia sumando ingresos**.

repository/SaleRepository.java - las dos condiciones

```
* utilidad. Filtrar por APPROVED <b>no</b> excluye las anuladas; hacen falta las
* dos condiciones, porque el estado del pago y el de la venta son ejes
* ortogonales a propósito.
*/
public interface SaleRepository extends JpaRepository<SaleEntity, Long> {
    List<SaleEntity> findByCashRegister(CashRegisterEntity cashRegister);

    //Tendencia por día
    @Query("""
        SELECT CAST(s.saleDate AS date), SUM(s.total), COUNT(s)
        FROM SaleEntity s
        WHERE s.paymentStatus = :status AND s.cancelled = false AND s.saleDate >= :from AND s.saleDate < :to
        GROUP BY CAST(s.saleDate AS date)
        ORDER BY CAST(s.saleDate AS date)
    """)
```

Son dos ejes ortogonales a proposito: el estado del pago (¿se cobro?) y el estado de la venta (¿el cliente se llevo la comida?). Una venta con pago aprobado puede estar anulada. Por eso hacen falta las **dos** condiciones, y por eso el arreglo fue agregar AND s.cancelled = false a las 6 consultas de agregados.

El numero lo escribe el vendedor al abrir la caja, es obligatorio y es unico. Se decidio que se mantenga la regla de **una sola caja activa** y que **no** se agregara user_id: la caja se identifica por numero, no por quien la abrio.

service/impl/CashRegisterImpl.java - open() y normalizeNumber()

```
public CashResponse open(OpenCashRequest request){
    repository.findByActiveTrue().ifPresent(c -> {
        throw new CashException("Ya existe una caja abierta", HttpStatus.CONFLICT);
    });

    //V3: el numero que escribe el vendedor es OBLIGATORIO y UNIQUE.
    //Es lo que despues permite "filtrar por caja" en Reportes. Se valida a
    //mano (no solo con el UNIQUE de la BD) para dar un 409 con mensaje claro
    //en vez de un 500 de constraint.
    String number = normalizeNumber(request.getNumber());

    if(repository.existsByNumber(number)){
        throw new CashException(
            "Ya existe una caja con el numero \"" + number + "\". "
            + "Cada caja necesita un numero unico para poder filtrar los reportes.",
            HttpStatus.CONFLICT);
    }

    CashRegisterEntity cash = new CashRegisterEntity();

    //Inicializar para evitar valores null
    cash.setCashSales(BigDecimal.ZERO);
    cash.setDebitSales(BigDecimal.ZERO);
    cash.setCreditSales(BigDecimal.ZERO);
    cash.setTotalSales(BigDecimal.ZERO);
    cash.setExpectedAmount(BigDecimal.ZERO);
    cash.setDifference(BigDecimal.ZERO);
    cash.setTotalTickets(0);

    cash.setNumber(number);
    cash.setOpenedAt(LocalDateTime.now());
    cash.setOpeningAmount(request.getOpeningAmount());
    cash.setActive(true);
    repository.save(cash);

    return mapper.toResponse(cash);
}

/**
 * Normaliza el numero de caja: recorta espacios y capitaliza.
 *
 * <p>Se capitaliza a proposito: "caja 1" y "CAJA 1" son la MISMA caja y
 * deben ser el mismo numero. Sin esto el UNIQUE de la BD los dejaria pasar
 * como dos cajas distintas (PostgreSQL compara con mayusculas y minusculas
 * distintas) y el reporte de esa caja mezclaria dos cortes reales.
 */
private String normalizeNumber(String number){
    if(number == null || number.isBlank()){
        throw new CashException("El numero de caja es obligatorio", HttpStatus.BAD_REQUEST);
    }
}
```

normalizeNumber() recorta y **capitaliza**, y eso no es cosmetico: "caja 1" y "CAJA 1" son la misma caja. Sin la capitalizacion, el UNIQUE de PostgreSQL las dejaria pasar como dos cajas distintas (compara con mayusculas y minusculas diferentes) y el reporte de esa caja mezclaria dos cortes reales.

El chequeo de duplicado se hace **en el servicio y no solo con el UNIQUE** de la base. La base lanzaria una violacion de constraint (HTTP 500, sin mensaje util); con existsByNumber se devuelve 409 con un mensaje que explica por que importa.

Los endpoints nuevos de caja

Endpoint	Permiso	Para que sirve
GET /api/local/cash/history	VER_CAJA	Lista de cajas para el selector de reportes
GET /api/local/cash/number/{number}	VER_CAJA	Ver una caja por su numero
GET /api/local/reports/cash/{cashId}	VER_REPORTES	Ventas y utilidad de esa caja

El historial usa VER_CAJA y no CORTE_CAJA a proposito: ver una lista de cajas es *consultar*, no hacer un corte de caja. Mezclar los dos permisos hubiera hecho que un cajero con permiso de corte no pudiera ver el selector.

El detalle por caja

CashReportDTO trae los importes del corte, la utilidad de esa caja y la lista de sus ventas con sus renglones. Va por cashId y no por numero a proposito: el numero es dato de entrada del usuario (puede venir con espacios o minusculas) y el id es la clave real. El frontend resuelve el numero al id usando el historial.

El costo de esta caja se calcula recorriendo los renglones en Java y no con una consulta agregada, porque el filtro aqui no es un rango de fechas sino la caja misma. Forzar el rango 'abierta a cerrada' dejaria fuera las ventas de una caja que sigue abierta.

6.1 Compras: el boton Confirmar

compras.html - columna Estado y acciones

```
<td>{{ c.totalItems }}</td>
<!-- class="money" -> alinea a la derecha con digitos tabulares (compras.css).
    | number:'1.2-2' formatea a 2 decimales (ej: 1500 -> 1,500.00). -->
<td class="money">{{ c.total | number:'1.2-2' }}</td>
<td>
  <span class="badge" [class.badge-confirmada]="c.confirmed" [class.badge-pendiente]="!c.confirmed">
    {{ c.confirmed ? 'Confirmada' : 'Pendiente' }}
  </span>
  <div class="sub" *ngIf="c.confirmed && c.confirmedAt">
    {{ c.confirmedAt | date: 'dd/MM/yyyy HH:mm' }}
  </div>
</td>
</td>
```

La tabla gana una columna **Estado** con dos badges: **Pendiente** en ambar y **Confirmada** en verde. La fila confirmada se atenúa. El boton **Confirmar** aparece solo con el permiso y se deshabilita si la compra ya esta confirmada; **Cancelar** tambien.

El dialogo de confirmar avisa de las dos cosas que van a pasar y de la que no se puede deshacer: entra la mercancia, se suma el stock y se guarda el costo, y despues la compra ya no se puede cancelar. El dialogo de cancelar, en cambio, dice 'todavia no tiene efecto en el stock', porque es verdad.

La fila se reemplaza por la **respuesta del servidor**, no por estado local optimista. El confirmedAt lo pone el backend y es la evidencia de cuando ocurrio; inventarlo en el cliente seria mentirle al usuario sobre su propia operacion.

6.2 Reportes: la utilidad y el filtro por caja

reportes.html - tarjetas de utilidad

```
<span class="summary-label">Ticket promedio</span>
<span class="summary-value">{{ summary.averageTicket | number:'1.2-2' }}</span>
<span class="summary-sub">promedio por venta</span>
</div>
<div class="card summary-card" *ngFor="let m of summary.paymentMethods">
  <span class="summary-label">{{ m.label }}</span>
  <span class="summary-value">{{ m.total | number:'1.2-2' }}</span>
  <span class="summary-sub">{{ m.count }} tickets</span>
</div>
</div>

<!-- V3: UTILIDAD (ingresos ? costo de lo vendido).
  Es el bloque que responde "¿estoy ganando?". El costo sale del costo
  CONGELADO en cada renglón de venta, no del total de compras del
  periodo; ese dinero no se perdió, quedó en el almacén. -->
<div class="summary-grid fade-in" *ngIf="profit">
  <div class="card summary-card accent-blue">
    <span class="summary-label">Ingresos</span>
    <span class="summary-value">{{ profit.revenue | number:'1.2-2' }}</span>
    <span class="summary-sub">{{ profit.tickets }} tickets válidos</span>
  </div>
  <div class="card summary-card accent-orange">
    <span class="summary-label">Costo de lo vendido</span>
    <span class="summary-value">{{ profit.costOfGoodsSold | number:'1.2-2' }}</span>
    <span class="summary-sub">lo que se llevaron los clientes</span>
  </div>
  <div class="card summary-card" [class.accent-green]="profit.grossProfit >= 0" [class.accent-red]="profit.grossProfit < 0">
    <span class="summary-label">Utilidad</span>
```

Tres tarjetas: **Ingresos**, **Costo de lo vendido** y **Utilidad**. La utilidad se pinta de **rojo** cuando es negativa, y es el unico acento rojo del reporte justamente por eso: cuando se perdio dinero, tiene que verse.

Debajo, si hay renglones sin costo, aparece un aviso que explica que la utilidad esta **sobrestimada**. Podria haberse omitido y mostrar el numero bonito; se decidio mostrarlo porque un numero que el usuario no puede explicar es peor que un numero feo.



6.3 Abrir caja

cobro.html - el input de numero

```
<!--  
  COMPONENTE COBRO (POS) - usa layout con sidebar  
  sidebar.menuOpen viene del SidebarService (singleton compartido).  
  [class.collapsed]="!sidebar.menuOpen" alterna la clase .collapsed en .layout  
  que dispara la transicion CSS: width 800ms ease-in-out en .sidebar.  
-->  
<div class="layout" [class.collapsed]="!sidebar.menuOpen">  
  <!--  
    Sidebar emite @Output() toggle con el nuevo estado (true/false).  
    El padre actualiza sidebar.menuOpen del servicio compartido.  
    Así el Sidebar y el padre SIEMPRE estan sincronizados.  
  -->  
  <app-sidebar (toggle)="sidebar.menuOpen = $event"></app-sidebar>
```

El numero es obligatorio y el backend responde 400 si falta, 409 si ya existe. El modal capitaliza y recorta antes de enviar, para que el usuario no tenga que acordarse de escribirlo en mayusculas.

20 tests nuevos. La suite completa queda en **173** (antes 153), toda en verde con `mvn test -Dtest='!ComprasApplicationTests'`.

PurchaseImplTest - la compra pendiente ni consulta los productos

```

}

@Test
void confirmarCompra_inexistenteLanza404(){
    when(purchaseRepository.findDetailedById(404L)).thenReturn(Optional.empty());

    assertThatThrownBy(() -> purchaseService.confirm(404L))
        .isInstanceOf(PurchaseException.class)
        .hasMessageContaining("no encontrada");
}

@Test
void confirmarCompra_dosRenglones_sumaTodosLosStocks(){
    ProductEntity leche = product(10L, "Leche", 20);
    ProductEntity cafe = product(11L, "Café 1kg", 2);

    PurchaseDetailEntity detLeche = new PurchaseDetailEntity();
    detLeche.setProduct(leche);
    detLeche.setQuantity(10);
    detLeche.setUnitCost(new BigDecimal("25"));

    PurchaseDetailEntity detCafe = new PurchaseDetailEntity();
    detCafe.setProduct(cafe);
    detCafe.setQuantity(4);
    detCafe.setUnitCost(new BigDecimal("40"));

    PurchaseEntity purchase = new PurchaseEntity();
    purchase.setId(9L);

```

Este test es el que blinda el diseño: exige que cancelar una compra pendiente **ni siquiera lea** los productos (`verify(productRepository, never()).findById(any())`). Si alguien reintrodujera un efecto en cancelar, el test falla. Es la versión positiva del bug del 2026-09-30.

PurchaseImplTest - confirmar dos veces no suma dos veces

```

// La garantía del diseño: una compra pendiente nunca tocó el
// inventario, así que cancelar no consulta NI ESCRIBE los productos.
verify(productRepository, never()).findById(any());
verify(productRepository, never()).save(any(ProductEntity.class));
assertThat(cafe.getStock()).isEqualTo(8);
}

@Test
void cancelarCompra_inexistenteLanza404(){
    when(purchaseRepository.findDetailedById(404L)).thenReturn(Optional.empty());

    assertThatThrownBy(() -> purchaseService.cancel(404L))
        .isInstanceOf(PurchaseException.class)
        .hasMessageContaining("no encontrada");
}

// =====
// CASO QUE CUBRE EL BUG ORIGINAL (2026-09-30), ahora por otra vía
// =====

/**

```

La idempotencia necesita un test propio. Sumar el stock dos veces por un doble clic es exactamente el tipo de error que nadie nota hasta que el inventario no cuadra, y para cuando se descubre ya no hay forma de saber desde cuando.

La regresión del bug de ventas anuladas

ReportQueryTest - ventaAnulada_noSumaIngresosAunqueElPagoEsteAprobado

```
// Ni en productos más vendidos: el Café solo aparece en ventas RECHAZADA y
// ANULADA, así que no debe existir como producto vendido
List<Object[]> top = saleRepository.findTopSellingProducts(
    from, to, PaymentStatus.APPROVED, PageRequest.of(0, 10));
assertThat(top).hasSize(1);
assertThat(top.get(0)[1]).isEqualTo("Gaseosa");
}

// =====
// COSTO DE LO VENDIDO (V3) -- la base de la utilidad
// =====

/**
 * El costo de lo vendido sale del unitCost CONGELADO en el renglón, y solo de
 * las ventas válidas. Aquí las dos ventas aprobadas no tienen costo (NULL),
 * así que el total es 0 y el contador de renglones sin costo es 2: el
 * reporte debe poder decir "no sé" en vez de inventar un 0 que finge una
 * utilidad de 25.00.
 */
@Test
void sumCostOfGoodsSold_sinCostosDevuelveCeroYCuentaLosDesconocidos() {
    List<Object[]> rows = saleRepository.sumCostOfGoodsSold(from, to, PaymentStatus.APPROVED);

    assertThat(new BigDecimal(rows.get(0)[0].toString()))
        .as("sin costo conocido, el costo de lo vendido es 0")
```

El truco está en el seed: se agregó una venta anulada de 500.00 con `paymentStatus = APPROVED`. Los tests que ya existían esperan 25.00 en el total, así que si el filtro `cancelled = false` desapareciera de cualquier consulta, el total saltaría a 525.00 y **cuatro tests distintos fallarían a la vez**. Es una regresión fijada con un solo dato en el seed, sin duplicar el test.

Ninguna de estas cosas rompe nada hoy, pero conviene escribirlas porque son las que se van a olvidar.



Verificacion final: 173/173 tests en verde, ng build sin errores y sin warnings nuevos. El unico test que no corre es ComprasApplicationTests, que es @SpringBootTest y exige una base de datos real.